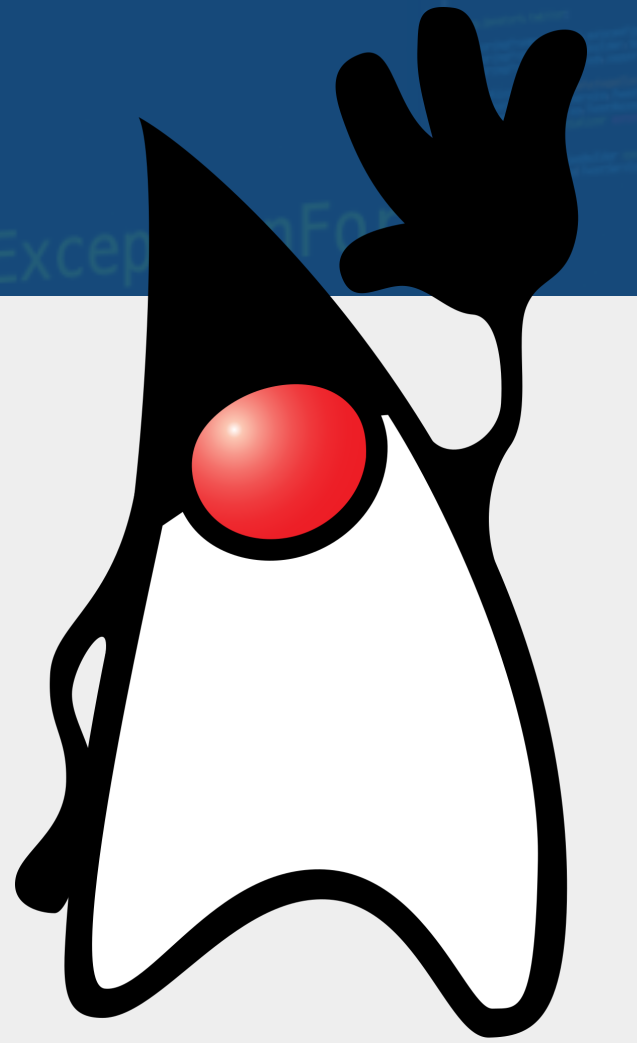


Java Programlama Dili

7. Bölüm

Karmaşık Tipler ve Değişkenler

Akın Kaldıroğlu



Amaç



- Bu bölümde Java'nın karmaşık tipleri ve değişkenleri ele alınacaktır

Konular



- Karmaşık Veri Tipleri
- Sınıf, Nesne ve Referans
 - null
- Ödevler
- Bundan Sonra?

Karmaşık Veri Tipleri



Karmaşık Veri Tipleri - I



- **Karmaşık veri tipleri (complex data types)**, ilkel tiplerin aksine tek parçadan oluşan atomik veriler değildir, birden fazla parçayı bir araya getirir
- Karmaşık veri tipleri, sınıflar ile oluşturulur
- Sınıflar sadece birden fazla veriyi bir araya getirmekle kalmaz, verinin yanına veri ile ilgili davranışları da koyar
- Buna **encapsulation** ya da **sarmalama** denir; bu konuya ileride detaylıca gireceğiz

Karmaşık Veri Tipleri - II



- Dolayısıyla nesne merkezli dillerde (object-oriented programming languages) karmaşık veri tipleri, veri ile o veri üzerinde çalışacak davranışları bir araya getiren yapılarla oluşturulur
- Dillerde farklılıklar olmakla birlikte bu yapılar Java dahil pek çok dilde sınıflar ile gerçekleştirilir

Sinif, Nesne ve Referans



Sınıf ve Nesne - I



- **Sınıf (class)**, kendisinden üretilecek olan nesnelerin şablonu ya da kalıbıdır
- **Nesne (object, instance)**, sınıfın çalışma zamanında bellekte oluşan örnekleridir
- Sınıf şablon ya da kalıp ise nesnelerini, o şablondan ya da kalıptan oluşturulmuş örnekler olarak düşünebiliriz

Sınıf ve Nesne - II



- Sınıflar ile **tipler** (**type**) oluşturulur
- İlkel tiplerden farklı olarak, istenildiği kadar sınıf, dolayısıyla yeni tip oluşturulabilir
- Aynı sınıftan üretilen nesnelerin tipleri aynıdır
 - Aynı sınıftan üretilen nesneler aynı özelliklere ve davranışlara sahiptir
 - Fakat nesnelerin özellikleri farklı değerler alabilir ve böylece davranışları da farklılaşabilir

Nesne ve Referans - I



- Nesneler, dinamik bellekteki (heap) adacıklar olarak düşünülürse, Java'da bu adacıklara uzanan/erişen **referanslar (reference)** vardır
- Referans yerine **tutaç, kulp (handle), işaretçi, gösterge (pointer)** vb. isimler de kullanılır
- Referanslar, bellekteki nesnenin adresini tutar (referencing an object)
- Kutu benzetmesini kullanılırsa, referans değişkenleri nesnenin adresini saklayan kutulardır

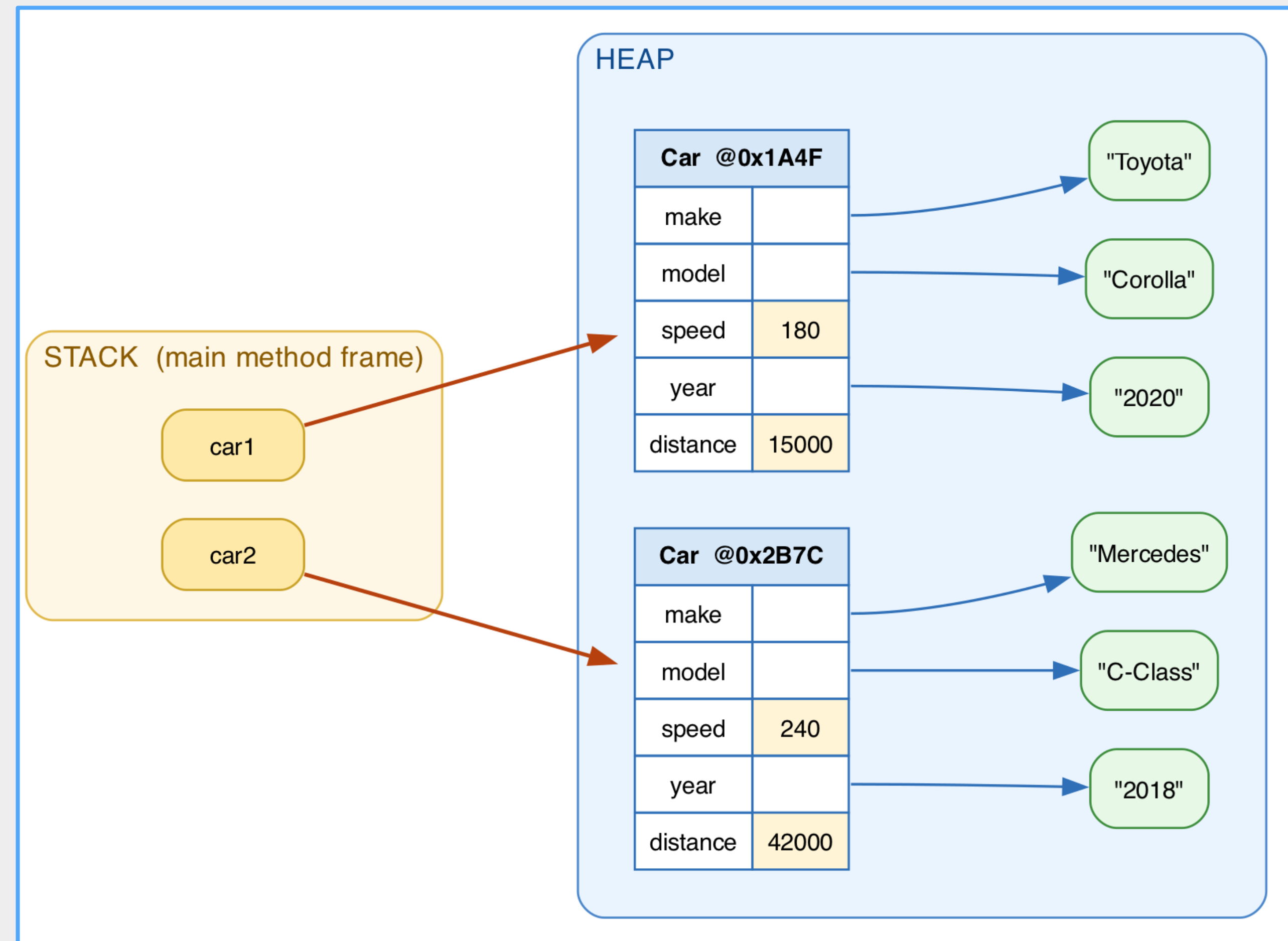
```
Car reference = new Car();
```

Nesne ve Referans - II



```
class Car{  
    String make;  
    String model;  
    String year;  
    int speed;  
    int distance;  
    ...  
}
```

```
class CarTest{  
    void main(){  
        Car car1 = new Car();  
        ...  
        Car car2 = new Car();  
        ...  
    }  
}
```



C++'ta Pointerlar



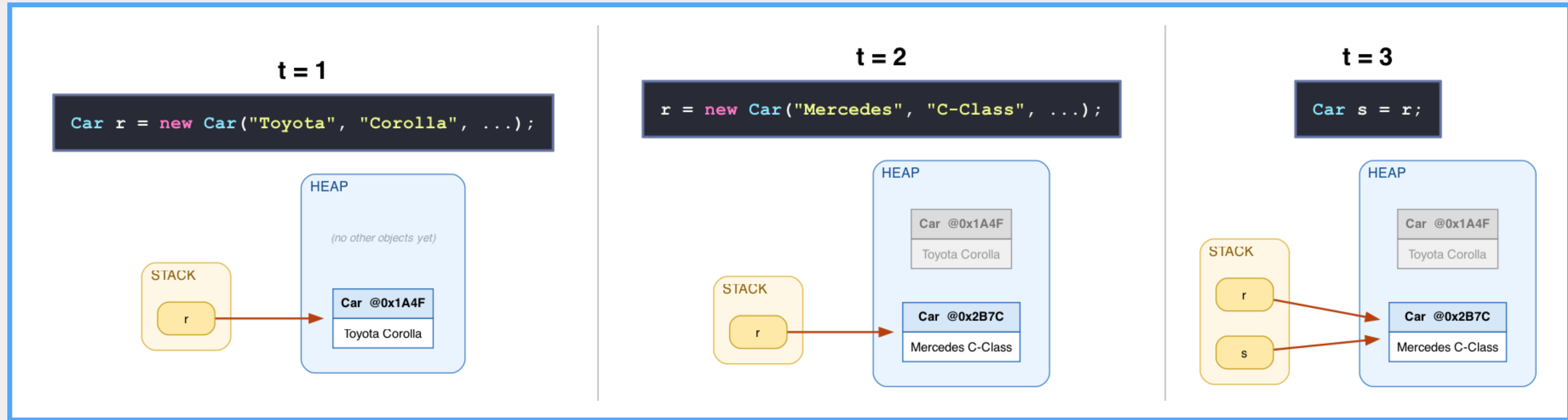
- C++'ta referansa pointer denir ve nesnenin bellekteki adresine pointer ile ulaşılır
- Bu adres üzerinden aritmetik işlemler de yapılabilir
 - Pointer aritmetiği (pointer arithmetic)
- Java'da, C++'taki anlamıyla pointer yoktur ya da Java'da pointer, referanstır ve referanslar nesnelere soyut erişim mekanizmalarıdır
 - Java'da nesnenin adresi bilinemez ve pointer aritmetiği de yoktur

Nesne ve Referans - III



- **Java'da nesnelere, sadece referanslarıyla erişilebilir**
- **Referanslar, nesnelere soyut erişim mekanizmasıdır**
 - Bir referans bir t anında, sadece bir nesneyi gösterebilir
- Bir nesnenin birden fazla referansı olabileceği gibi, bir referans farklı zamanlarda aynı tipten farklı nesneleri gösterebilir

Nesne ve Referans - IV



- Aslolan, referans ile nesnenin tiplerinin aynı olmasıdır
- Bunun detaylarına ileride gireceğiz

Referans Değişkeni (Reference Variable)

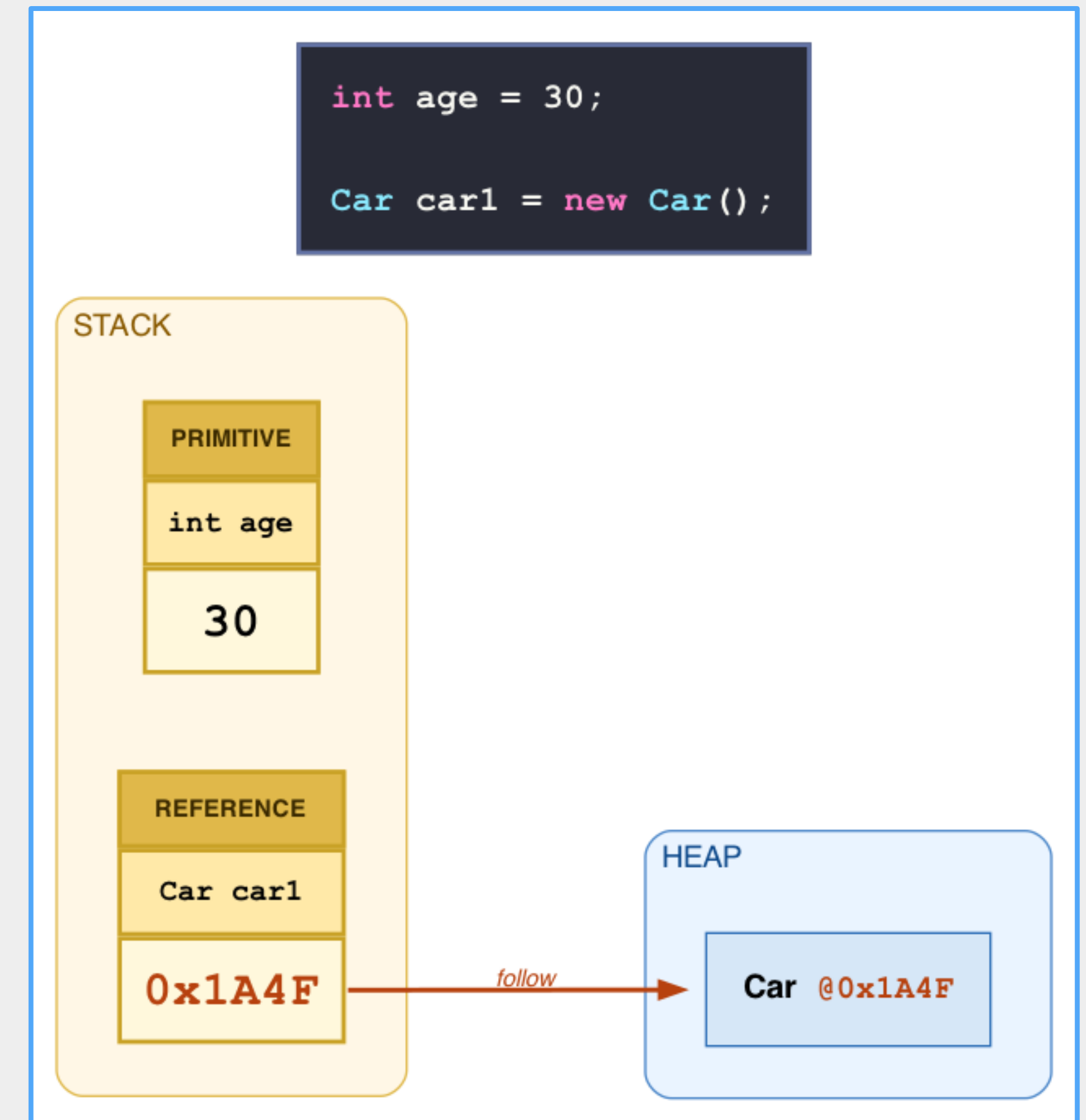


- Dolayısıyla, referanslar aynı zamanda değişkendir (variable)
 - **Referans değişkeni (reference variable)**
- Referansın tipi, gösterdiği nesnenin tipidir
- Referans değişkenine, **karmaşık değişken (complex variable)** de denir çünkü tipi karmaşıktır

İlkel ve Referans Değişkenler - I



- İlkel ve referans değişkenler arasındaki en temel fark, ilkel değişkenlerin sabite değerleri olurken, referans değişkenlerin değerinin, gösterdikleri nesne (gerçekte nesnenin adresi) olmasıdır
- Yani değişkenler kutu olarak düşünüldüğünde, ilkel değişkenler kutularında değeri saklarken referans değişkenleri kutularında nesnenin adresini saklar



İlkel ve Referans Değişkenler - II



- Bu farklılığı bir örnekle görelim
- Aşağıdaki örnekteki `j` ile `s1`, `s2` ve `tmp`'in değerinin ne olacağını düşünelim

```
int i = 5;  
int j = i;  
i = 7;
```

Bu satırda `j`'nin değeri nedir?

```
String s1 = new String("a");  
String s2 = new String("b");  
String tmp = s1;  
s1 = s2;  
s2 = tmp;
```

Bu satırda `s1` ve `s2`'nin değeri nedir?

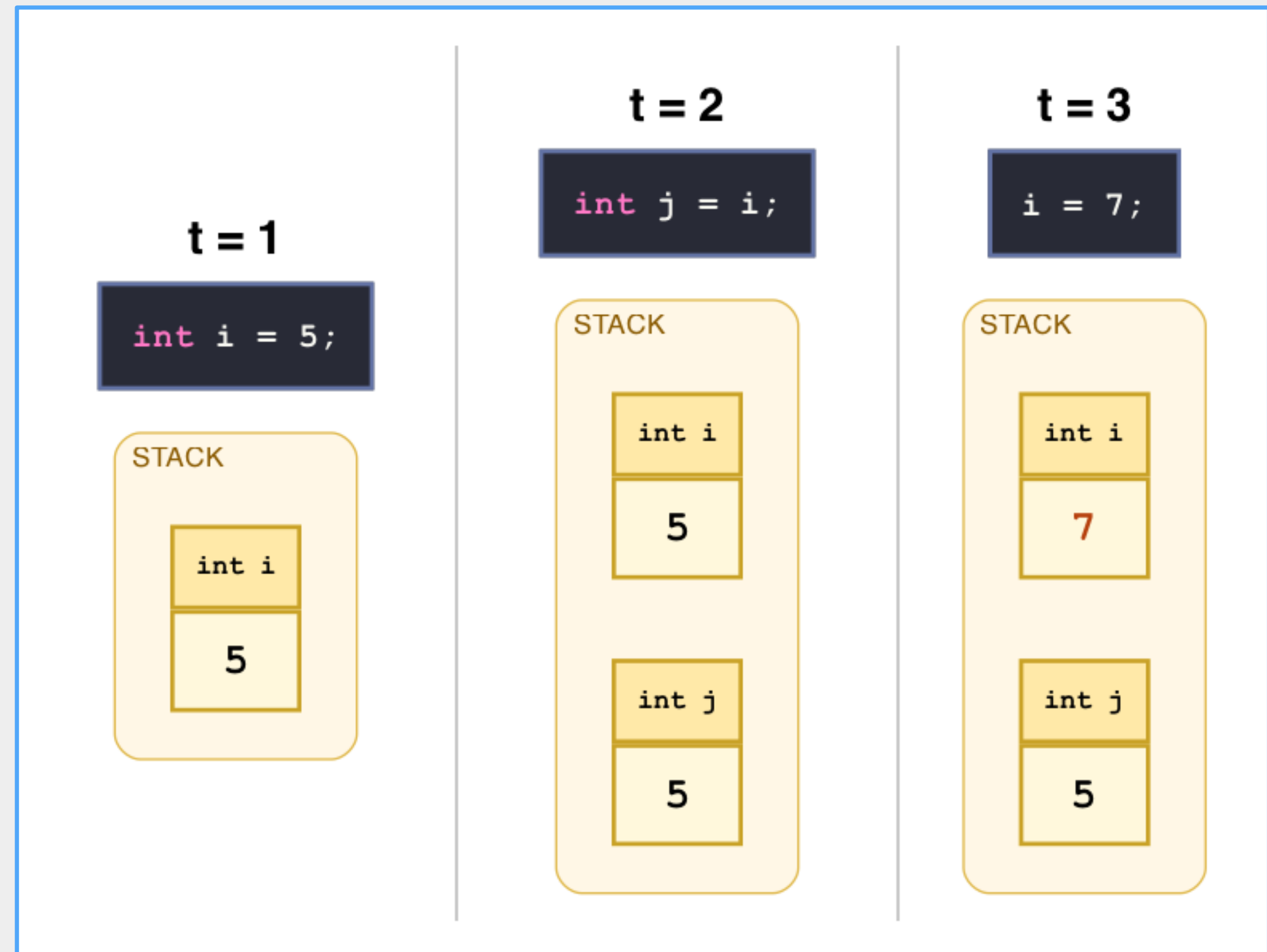
- Not: Java'da `new String("a")` kullanımı tercih edilmez. Detaylarına ileride gireceğiz.

İlkel ve Referans Değişkenler - II



```
int i = 5;  
int j = i;  
i = 7;  
// 5'tir
```

Bu satırda j'nin değeri nedir?

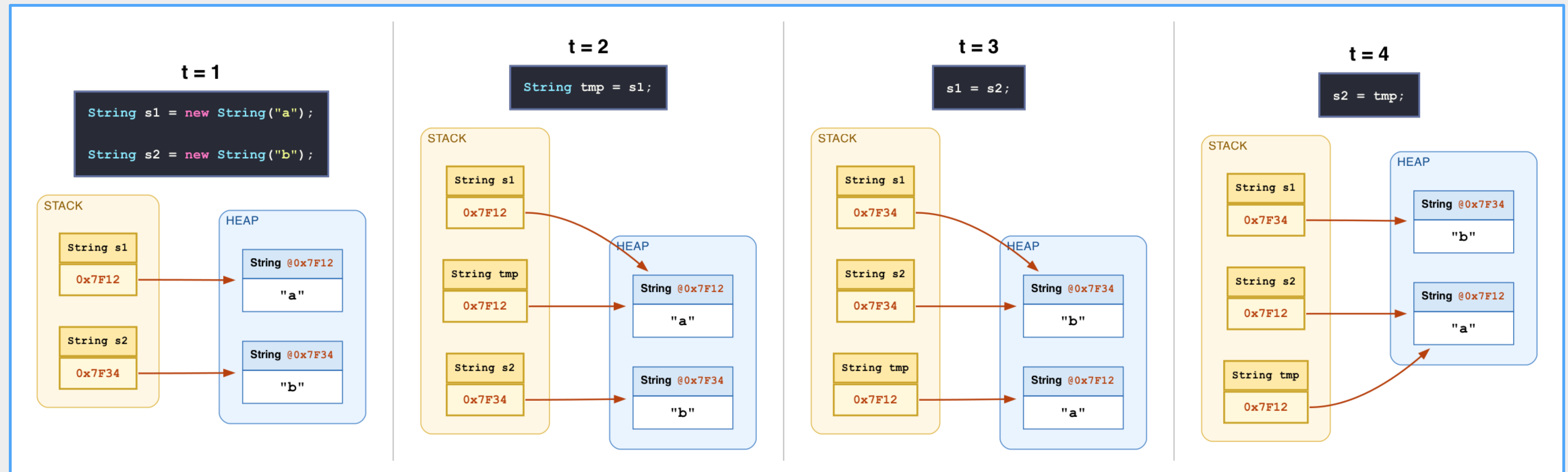


İlkel ve Referans Değişkenler - IV



```
String s1 = new String("a");  
String s2 = new String("b");  
String tmp = s1;  
s1 = s2;  
s2 = tmp;
```

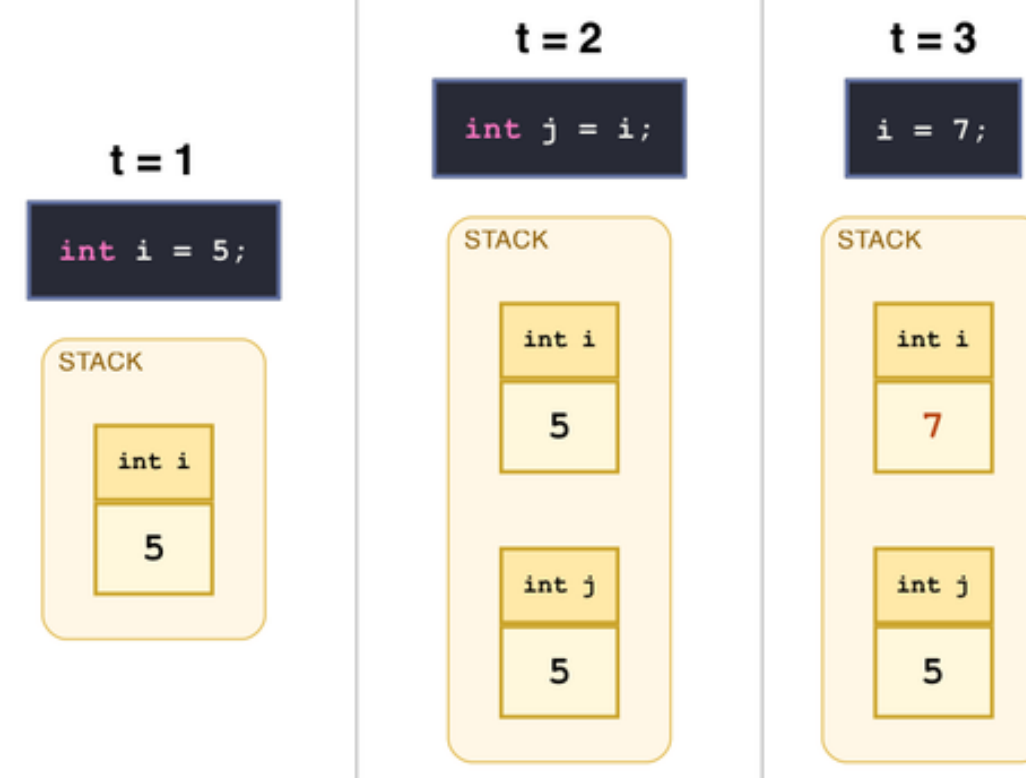
Bu satırda s1 ve s2'nin değeri nedir?



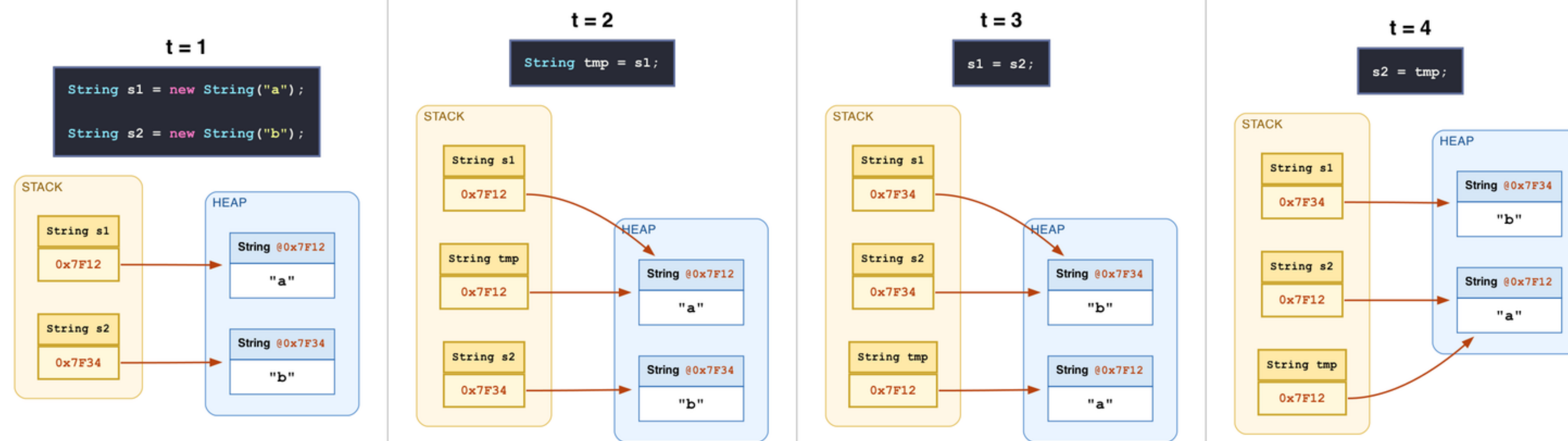
İlkel ve Referans Değişkenler - V



PRIMITIVE COPY — values copy independently



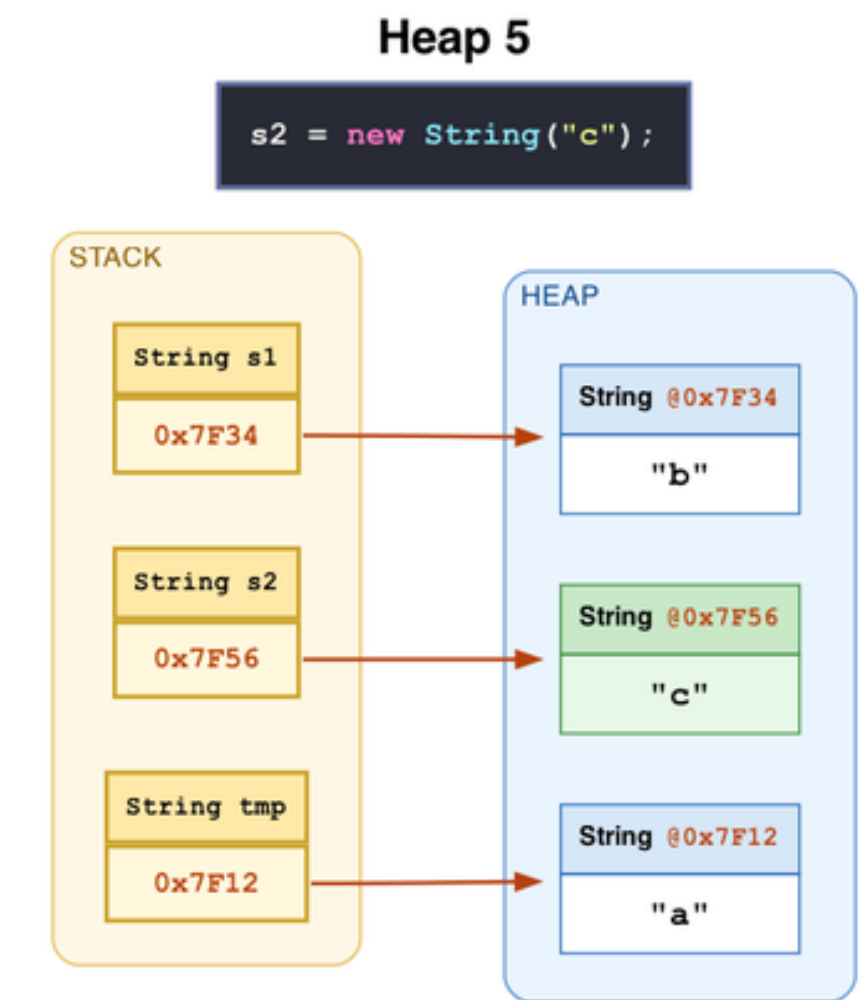
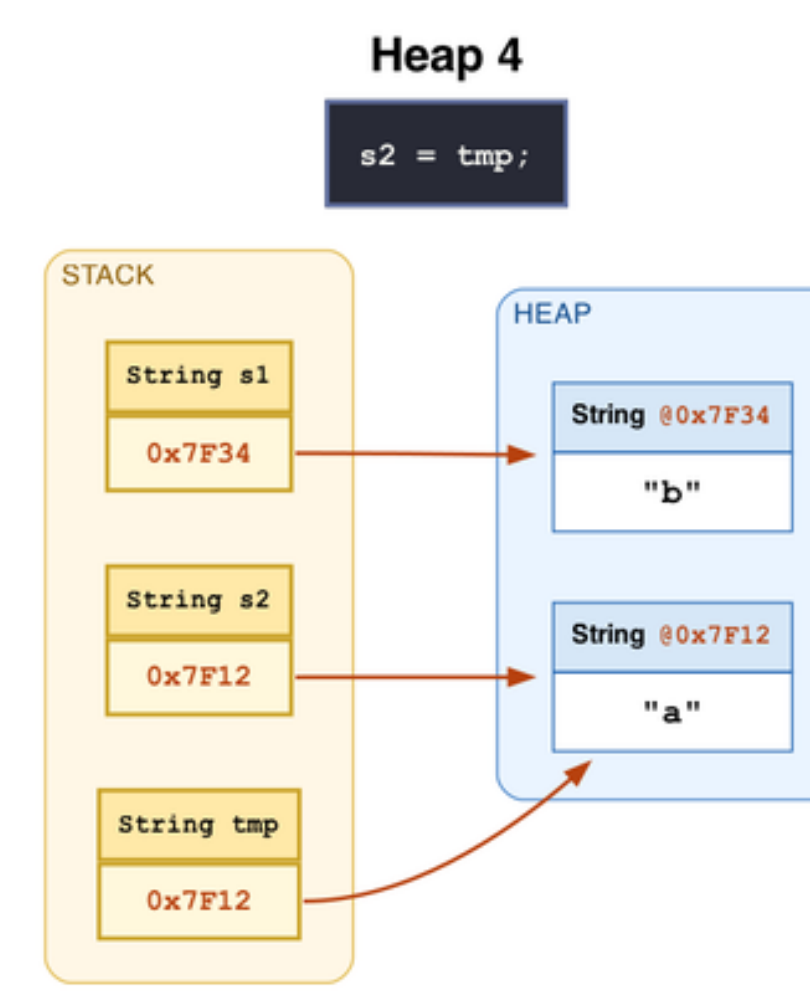
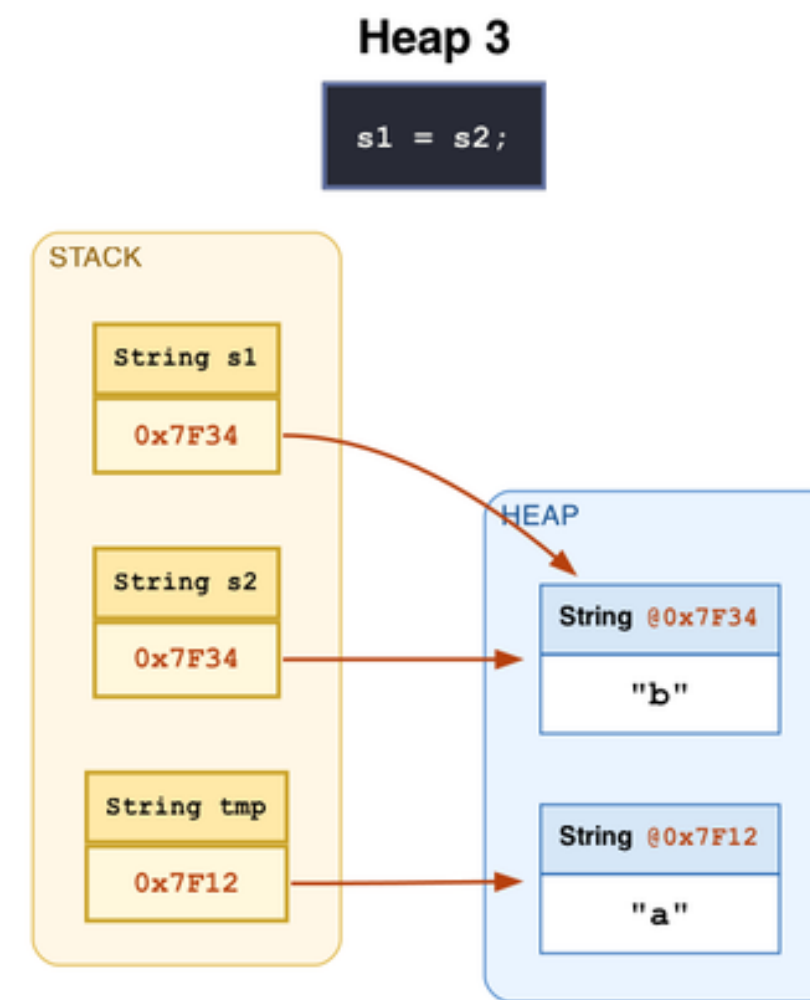
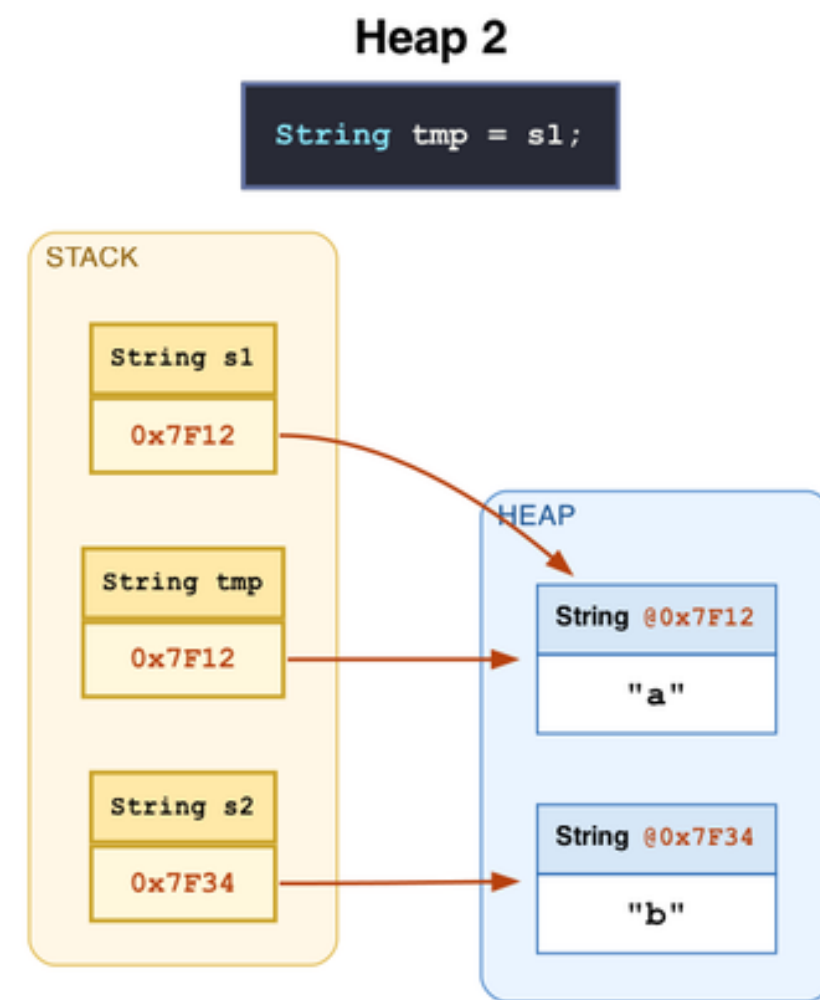
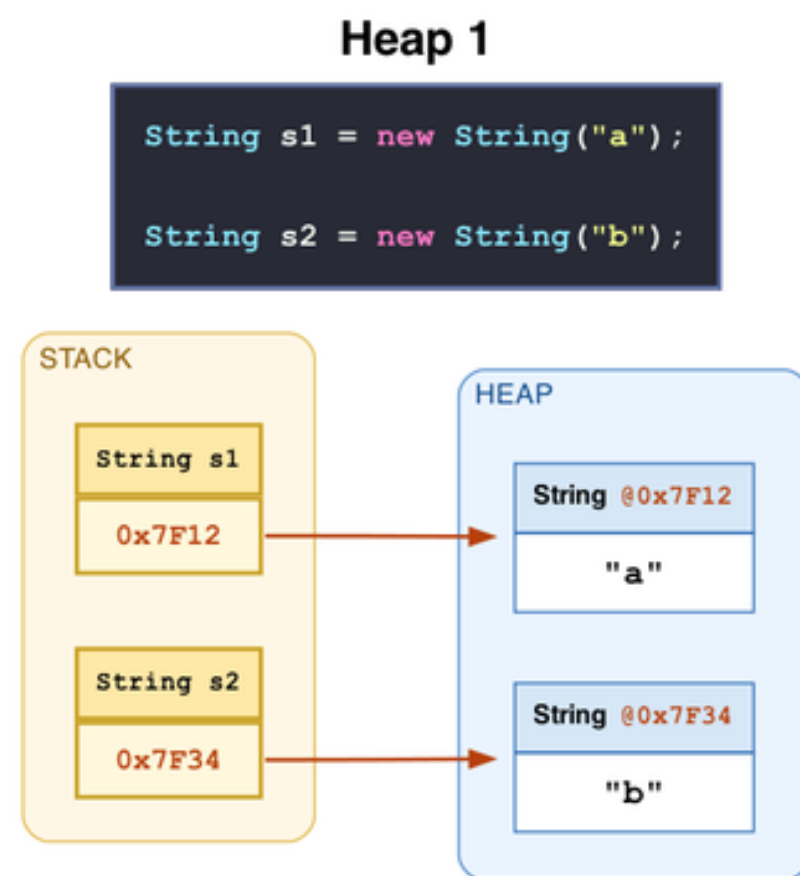
REFERENCE SWAP — addresses copy; heap objects don't move



İlkel ve Referans Değişkenler - VI



```
// Heap 1
String s1 = new String("a");
String s2 = new String("b");
// Heap 2
String tmp = s1; // tmp ⇒ "a"
// Heap 3
s1 = s2;          // s1 ⇒ "b"
// Heap 4
s2 = tmp;         // s2 ⇒ "a"
// Heap 5 reassigning s2
s2 = new String("c"); // s2 ⇒ "c"
```



İlkel ve Referans Değişkenler - VII



- İlkel değişkenler kendilerine atanan değerleri kopyalarlar, kendilerinde tutarlar ya da kendi kutularına koyarlar
- Böylece ilkel değişken, atanan değere verilen bir isim haline gelir
- Ama aynı değer pek çok ilkel değişkene atanabilir, kutusuna konabilir
- Dolayısıyla $j = i$ 'de, i 'nin değeri j 'ye kopyalanır ve j , i 'nin değerine sahip olur
- Sonrasında i ve j bağımsız olarak farklı değerler alabilirler, aralarında hiçbir bağ ya da ilişki yoktur

İlkel ve Referans Değişkenler - VIII



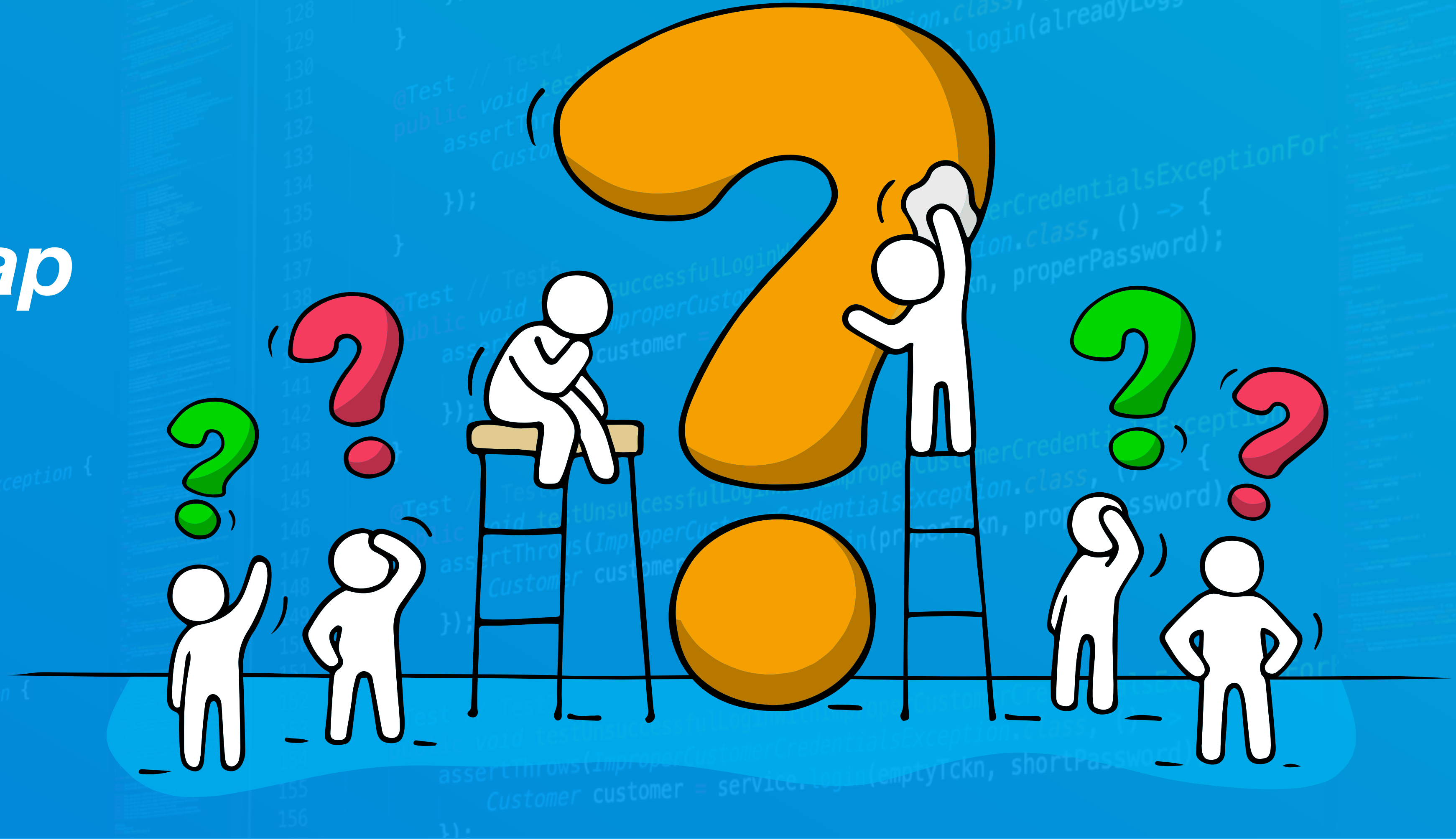
- Ama referanslarda durum farklıdır, referanslar ise kutularında gösterdikleri nesnelerin adreslerini tutarlar
- $s1 = s2$ ifadesi ile $s1$, $s2$ 'nin değerine sahip olur, yani $s2$ 'in gösterdiği adres $s1$ 'e kopyalanır ve böylece ikisi de aynı nesneyi gösterir hale gelir
- Bu sebeple, bir referansın gösterdiği nesnenin durumunda değişiklik olduğunda, o nesneyi gösteren tüm referanslar üzerinden bu değişiklik görülür

PrimitivesAndReferences



- `PrimitivesAndReferences`
- Bu örnekte ilkel tiplerle olan davranış farklılığı ele alınacaktır
- Ayrıca referans ile nesne arasındaki ilişki gözlemlenecektir

Soru ve Cevap Zamanı!





null

null - I



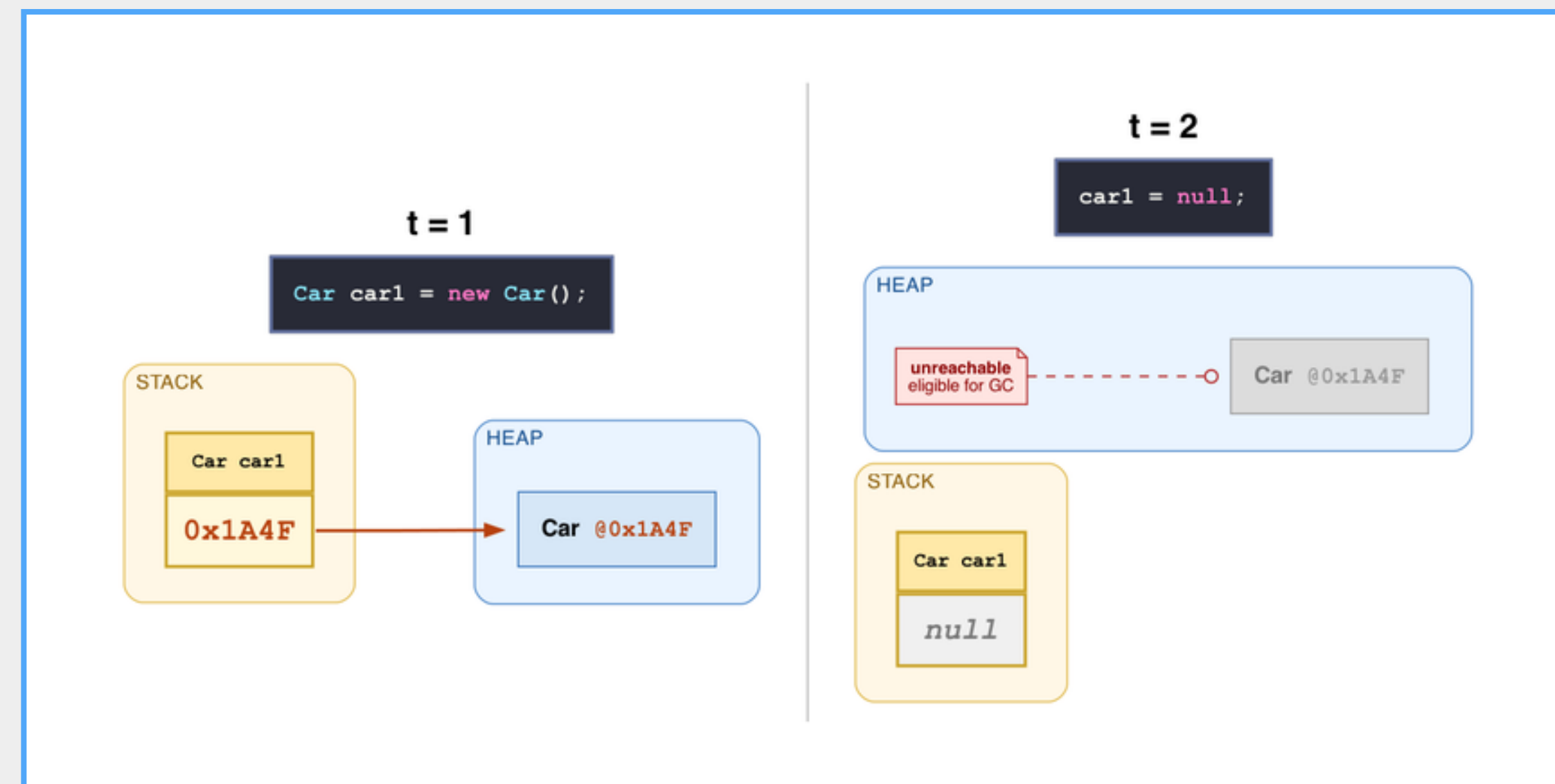
- Bir referans, herhangi bir nesneyi göstermek, bir başka deyişle kutusunda bir adres olmak zorunda değildir; eğer bir referans hiçbir nesneyi göstermiyorsa bu durumda referans **null**'dir, boştur
- Bir nesneyi gösteren referansa **null** atanırsa, o referans ile nesnesi arasındaki ilişki ortadan kalkar ve referans **null** olur
- Ama nesneyi gösteren başka referanslar var olmaya devam edebilir
- **null**, sadece referans değişkenlerine atanabilir, ilkel değişkenlere atanamaz

null - II



- Hiçbir referansı olmayan nesneye **kayıp nesne (lost object)** denir
- Kayıp nesnelere erişilemez (unreachable), çünkü onu gösteren hiçbir referans yoktur

```
Car car1 = new Car();  
...  
car1 = null; //car1 is null
```



null - III



- **null** olan bir referansa ulaşılmaz, çünkü, **null** olan referans hiçbir nesneyi göstermez, dolayısıyla bir alanına ya da davranışına erişilebilecek nesne yoktur
- **null** olan referansa erişilmesi halinde `NullPointerException` sıra dışı durumu fırlatılır

```
car1 = null;  
car1.go(50); // NullPointerException  
// Cannot invoke "...Car.go(int)" because "car1" is null
```

- Dolayısıyla bir referans ile bir nesnenin bir alanına ya da davranışına erişmeden önce o referansın **null** olmadığından emin olun

null - IV



- Dillerde bir referansın `null` olmasına rağmen `NullPointerException` sıra dışı durumu fırlatmasının önüne geçmek için farklı mekanizmalar veya uygulamalar vardır
- Referansın `null` olmadığından emin olunmadığında, en basiti `null` kontrolü yapmaktır

```
car1 = ...  
...  
car1.go(50); //NullPointerException
```

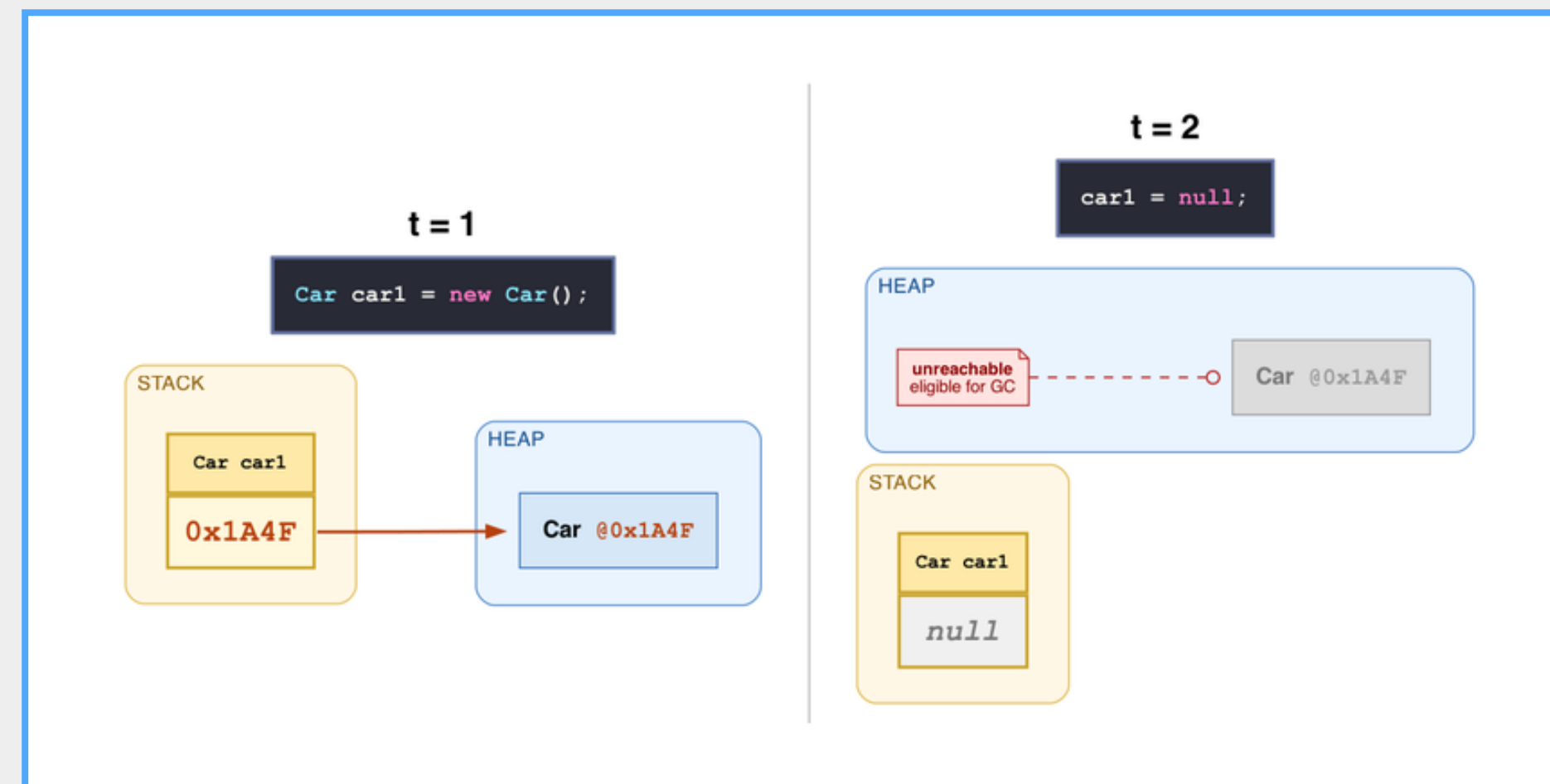
```
if(car1 != null)  
    car1.go(50); //No problem
```

null - V



- Kullanılmadığı halde bellekte kalmaya devam eden nesneler (memory leak) bellek sızıntısı oluşturur.
- Bellek sızıntısından kaçınmak için, kayıp nesnelerin kullandıkları bellek, **Çöp Toplayıcı (Garbage Collector)** tarafından toplanır, sisteme geri kazandırılır: **automatic memory management**

```
Car car1 = new Car();  
...  
car1 = null; //car1 is null  
//Car object will be collected by GC
```





- `car.CarTest`
- Bu örnekte `Car` sınıfından oluşturulan nesnelerin özellik ve davranışları gözlemlenecektir

CarReferences - I



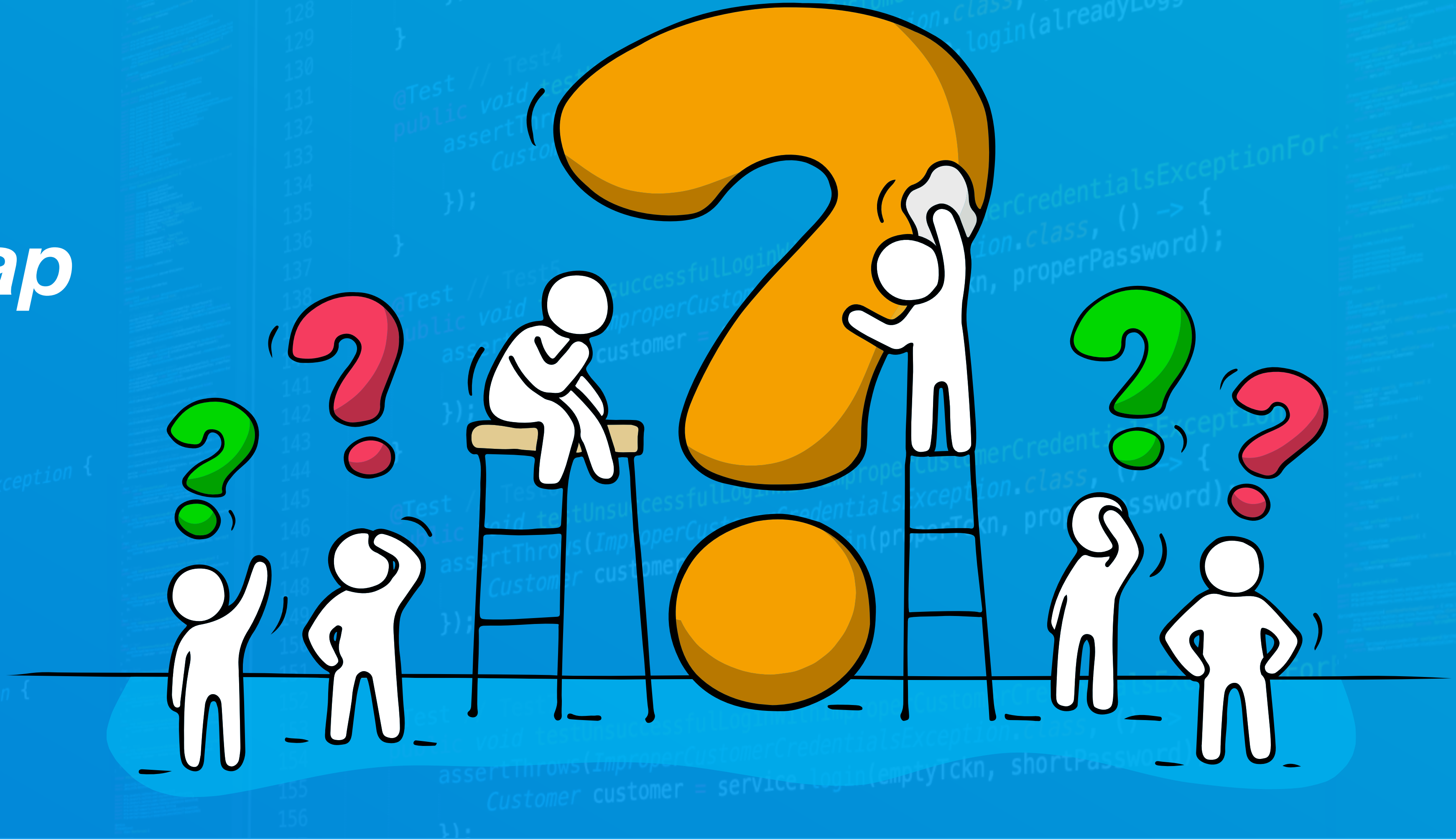
- `car.CarReferences`
- Burada referans ile nesne arasındaki ilişki ve **null** referansın davranışı incelenecektir
- Aynı nesneye birden fazla referans olduğu durumda, referansların birinin yaptığı değişikliğin, diğer referansın üzerinden gözlemlenebildiğini de inceleyin

CarReferences - II



- `car.CarReferences`
 - Bir `Car` referansını `null` yapıp doğrudan referansı print edin
 - Bir `Car` referansını `null` yapıp üzerindeki bir özelliğe ya da davranışa ulaşmaya çalışın, ne olduğunu gözlemleyin
 - `CarFactory` yardımıyla elde edilen ve `null` olma ihtimali olan bir `Car` referansı üzerinde güvenli bir işlem yapabilmek için nasıl bir önlem alınması gerektiğini gözlemleyin

Soru ve Cevap Zamanı!



Ödevler





1. Aşağıdaki değişkenlerin ilkel ya da karmaşık olduğuna karar verin

```
int age = 30;  
double weight = 75.5;  
String name = new String("Java");  
char grade = 'A';  
boolean isActive = true;  
Car myCar = new Car();
```



2. Aşağıdaki kod parçasının çıktısı ne olur?

```
String x = new String("Apple");  
String y = x;  
String z = x;  
x = new String("Banana");  
  
System.out.println(x);  
System.out.println(y);  
System.out.println(z);
```



3. `Car` sınıfında `color` isimli bir alanın ve ilgili `setColor()` metodunun olduğunu farzedin. Aşağıdaki kod parçasının çıktısı ne olur?

```
Car c1 = new Car();  
c1.setColor("Red");  
Car c2 = c1;  
c2.setColor("Blue");  
System.out.println(c1.getColor());
```



4. Aşağıdaki kod parçası çalıştığında bellekte kaç tane **Car** nesnesi oluşur ve bunlardan kaç tanesi erişilebilir halde olur?

```
Car a = new Car();  
Car b = new Car();  
a = b;
```



5. Aşağıdaki kod parçası çalıştığında ne basar? Ayrıca **c1** ve **c2** referansları neyi göstermektedir?

```
Car c1 = new Car();  
Car c2 = c1;  
c1 = null;  
c1.make = "Mercedes";  
System.out.println(c1.make);  
System.out.println(c2.make);
```



6. `car.CarReferences` sınıfındaki `main` metotta 2 tane `Car` nesnesi oluşturun ve değişkenlerine uygun değerler atayın ve üzerinde metotlar çağırın
 - a. Çağırduğunuz metotlardan sonra nesnelerin `getInfo()` metodunu da çağırın ve sonucu yazdırın
 - b. `Car` nesnelerinin referanslardan birine `null` atayın ve üzerinde metot çağırın, sonucu gözlemleyin ve sonrasında bunu önleyin

Bundan Sonra?



Bundan Sonra?



- Şimdi ise bir sonraki konuda ilkel olsun karmaşık olsun, değişkenlerin ortak özelliklerini ele alacağız

Bölüm Sonu

Soru ve Cevap Zamanı!

